

Corridor Módulo 1.0 – Visão Geral do Ecossistema ,

totalmente estruturado, limpo e profissional, no formato **Markdown modular**, pronto para uso em documentação oficial.

MÓDULO 1.0 — VISÃO GERAL DO ECOSSISTEMA

(documentação modular em Markdown)

1.0 Visão Geral do Ecossistema

1.1 Propósito do Sistema

O Ecossistema Digital do Corredor do Lobito é uma plataforma governamental e empresarial integrada para:

- Regular operações comerciais transfronteiriças.
- Garantir transparência e rastreabilidade de produtos, pedidos e operações logísticas.
- Controlar preços, licenças, impostos e políticas fiscais.
- Fornecer ferramentas de auditoria, monitoramento e compliance.
- Unificar governo, empresas, operadores logísticos e compradores numa única infraestrutura digital.

A plataforma suporta operações B2G, B2B e G2G.

1.2 Escopo Principal

O ecossistema cobre os seguintes domínios funcionais:

1. ****Licenciamento de empresas****
2. ****Cadastro e publicação de produtos****
3. ****Pedidos e transações****
4. ****Logística e embarques****
5. ****Fiscalidade e tarifas****
6. ****Price proposals (base oficial de preços)****
7. ****Relatórios fiscais, operacionais e de auditoria****
8. ****Compliance e investigação de anomalias****
9. ****Autenticação governamental (SSO/OIDC + 2FA obrigatório)****

1.3 Arquitetura de Governança (Multicamadas)

A governança é estruturada em papéis com níveis hierárquicos e responsabilidades distintas:

****STATE**** (Autoridade Governamental Máxima)

Define políticas e toma decisões críticas:

- Aprova licenças, produtos, price proposals, relatórios.
- Bloqueia/cancela pedidos e empresas.
- Aprova políticas fiscais.
- Tem acesso total de leitura e auditoria.

◆ **STAFF** (Executores Governamentais)

Implementam as decisões do STATE:

- Validam documentação tecnicamente.
- Executam operações autorizadas pelo STATE.
- Monitoram o ecossistema.
- Suporte operacional.
- NÃO tomam decisões estratégicas.

◆ **COMPLIANCE**

Órgão independente de fiscalização:

- Audita operações, transações, logs e políticas.
- Analisa e detecta anomalias.
- Gera relatórios de compliance.
- Pode bloquear transações suspeitas.

◆ **SPECIALIST / ANALYST**

Perfis técnicos estratégicos:

- Criam price proposals.
- Produzem relatórios fiscais e operacionais.
- Realizam análises de mercado.
- Investigam padrões suspeitos.

◆ **COMPANY / BUYER / PRODUCER / OPERATOR**

Perfis empresariais regulados:

- Empresas: Licenciamento + catálogo + gestão de pedidos.
- Buyers: Criam pedidos e transações.
- Producers: Criam produtos e submetem para publicação.
- Operators: Gerem embarques e tracking.

1.4 Visão dos Componentes do Ecossistema

✅ **Cadastro e Licenciamento**

Inclui criação de empresas, envio de documentos, validação técnica (STAFF) e decisão de aprovação (STATE).

✅ **Gestão de Produtos**

- Producer cria → Analyst revisa → STATE aprova → Produto publicado.
- Metadados incluem histórico de preço, origem, categoria e atributos fiscais.

✅ **Price Proposals**

Base central de formação de preços oficiais:

- Criadas por SPECIALIST/ANALYST.
- Avaliadas pelo STATE.
- Versionamento e snapshots imutáveis.

✅ **Pedidos e Transações**

- Buyers criam pedidos.
- Cálculo de impostos, tarifas e políticas cross-border automático.
- Transações auditáveis.

✅ **Logística e Embarques**

- Operadores registram eventos de embarque.
- Customs valida documentos aduaneiros.
- Shipping engine integra tracking.

✅ **Fiscalidade e Tributação**

Inclui:

- Taxes
- Tariffs
- Quotas (se aplicável)
- Regras fiscais binacionais

 ****Compliance & Auditoria****

- Logs imutáveis.
- Análise de riscos.
- Detecção de anomalias.
- Auditorias periódicas.

 ****Dashboards Governamentais****

KPIs:

- Operacionais
- Fiscais
- Compliance
- Licenciamento
- Logística

1.5 Princípios Arquiteturais do Sistema

****1. Governança Distribuída****

Os estados membros atuam como autoridades soberanas na aprovação e revisão de processos.

****2. Transparência Total****

Logs, versionamento e snapshots imutáveis garantem rastreabilidade.

****3. Segurança de Alto Nível****

- SSO/OIDC com GovID
- 2FA obrigatório
- Claims específicos por role
- Políticas de acesso rígidas

****4. Modularidade****

Cada domínio funciona como um módulo independente porém interoperável.

****5. Baseado em Workflows****

Cada operação segue fluxos formais, auditáveis e padronizados.

****6. Interoperabilidade Transfronteiriça****

Regras fiscais, tarifas e tracking funcionam entre vários países do corredor.

7. Observabilidade Completa

- Logs
 - Relatórios
 - Dashboards
 - Alertas de compliance
-

1.6 Stakeholders do Ecossistema

Stakeholder	Tipo	Responsabilidade Principal
STATE	Governo	Decisão estratégica
STAFF	Governo	Execução e suporte
COMPLIANCE	Auditoria	Fiscalização
SPECIALIST	Governo	Análises e propostas
ANALYST	Governo	Relatórios e dados
COMPANY	Empresa	Operações comerciais
PRODUCER	Empresa	Cadastrros de produtos
BUYER	Empresa	Pedidos e pagamentos
OPERATOR	Logística	Embarques e tracking
CUSTOMS	Aduana	Validação documental

1.7 Objetivos Funcionais do Ecossistema

Para Governo

- Controle fiscal centralizado.
- Supervisão em tempo real.
- Auditoria completa.
- Aplicação de políticas comerciais.

Para Empresas

- Acesso seguro a mercados regionais.
- Redução de burocracia e tempo.
- Transparência nas operações.
- Regras claras para preços, licenças e compliance.

Para Operadores Logísticos

- Melhor sincronização entre países.
- Eliminação de inconsistências documentais.
- Visibilidade total do fluxo.

1.8 Fluxo Geral do Ecossistema (Resumo)

Empresa → STAFF → STATE → Publicação → Mercado → Buyer → Pedido → Shipping
→ Compliance → Auditoria → Relatórios

1.9 Tecnologia e Integração (Visão High Level)

- **Backend:** Node.js / Strapi
- **Frontend:** Next.js / React
- **Autenticação:** Clerk + SSO Governamental
- **Banco de dados:** PostgreSQL
- **Integrações externas:** Operadores, Alfândega, Pagamentos
- **Auditoria:** Logs imutáveis + snapshots
- **Observabilidade:** Dashboards e relatórios governamentais

1.10 Conclusão

Este módulo apresenta uma visão completa do ecossistema, incluindo sua arquitetura, entidades, governança, objetivos e módulos funcionais. Servirá como base para os próximos capítulos detalhados envolvendo:

- Regras de negócios
- Workflows completos
- API detalhada
- Permissões e papéis
- Estruturas de dados

 **Módulo 1.0 concluído.**

MÓDULO 2.0 — ROLES E RESPONSABILIDADES, totalmente estruturado, claro e profissional, em Markdown, seguindo a mesma arquitetura modular.

MÓDULO 2.0 — ROLES E RESPONSABILIDADES

(documentação modular em Markdown)

2.0 Roles e Responsabilidades

2.1 Introdução

Este módulo define, de forma precisa, os papéis (roles) que compõem o ecossistema e determina:

- Responsabilidades formais de cada role.
- Poderes e limitações.
- Hierarquia de autoridade.
- Áreas de atuação.
- Ações permitidas em cada módulo.
- Escopo de segurança e auditoria.

A compreensão desses papéis é fundamental para implementar:

- Políticas de acesso,
- Segurança,
- Workflows,
- Permissões Strapi,
- Auditoria,
- Compliance.

2.2 Estrutura Hierárquica de Papéis

A hierarquia é governamental e empresarial, com níveis de autoridade definidos:

ADMIN (Superusuário)

- └─ STATE (Autoridade Governamental Máxima)
- └─ STAFF (Executores)
- └─ SPECIALIST (Técnico Analítico)
- └─ ANALYST (Técnico Operacional)

CUSTOMS (Aduana)

COMPANY (Empresa)

- └─ PRODUCER (Fornecedor)
- └─ BUYER (Comprador)
- └─ OPERATOR (Logística)

Todos os papéis são auditados por ****COMPLIANCE****.

2.3 Visão Geral dos Roles

ROLE	Tipo	Poder Principal	Nível de Acesso
ADMIN	Governamental TI	Configura o sistema	Total
STATE	Governo	Aprovação, decisão, auditoria	Alta
STAFF	Governo	Execução e monitoramento	Média
SPECIALIST	Governo	Propostas de preço e análises	Média
ANALYST	Governo	Relatórios e dados	Baixa
COMPLIANCE	Auditoria	Fiscalização e investigação	Alta
CUSTOMS	Aduana	Validação de documentos	Média
COMPANY	Empresa	Gestão operacional	Média
BUYER	Empresa	Compras	Baixa
PRODUCER	Empresa	Cadastro de produtos	Média
OPERATOR	Logística	Embarques e tracking	Média

2.4 Definição Completa de Cada Role

2.4.1 ADMIN (Superusuário Técnico)

Resumo

Papel máximo responsável pela infraestrutura, módulos, plugins, roles e permissões.

Responsabilidades

- Gerenciar configurações do sistema.
- Criar roles e permissões.
- Criar/editar/deletar usuários de qualquer role.
- Configurar integrações SSO/OIDC.
- Gerenciar bancos, clusters, backups e segurança.
- Criar políticas automatizadas.
- Auditar falhas técnicas.

Limitações

- Não aprova licenças.
- Não interfere em decisões governamentais.
- Qualquer remoção de usuários STATE/STAFF precisa de:
 - Proposta ADMIN
 - Validação COMPLIANCE

2.4.2 STATE (Autoridade Governamental Máxima)

Resumo

É o "governo dentro do sistema". Possui poder de decisão final.

Responsabilidades Principais

1. **Licenças**
 - Aprovar / rejeitar / suspender empresas.
2. **Produtos**
 - Aprovar publicação oficial.
3. **Pedidos**
 - Bloquear / cancelar pedidos suspeitos.

4. ****Price Proposals****
 - Aprovar propostas enviadas por SPECIALISTS.
5. ****Fiscalidade****
 - Criar e aprovar políticas fiscais (taxes, tariffs).
6. ****Compliance****
 - Revisar auditorias e tomar decisões.
7. ****Relatórios****
 - Publicar relatórios estratégicos.

****Poderes Especiais****

- Acesso total em leitura.
- Pode ordenar execução ao STAFF.
- Pode bloquear empresas e pedidos.

****Limitações****

- Não pode manipular dados técnicos.
- Não cria usuários.

****2.4.3 STAFF (Executores Governamentais)****

****Resumo****

Executa operações que o STATE decide, valida documentos e monitora processos.

****Responsabilidades****

- Validar documentação das empresas (conformidade técnica).
- Encaminhar para STATE decidir.
- Executar políticas aprovadas pelo STATE.
- Resolver tickets técnicos.
- Monitorar conformidade diária.
- Emitir relatórios de monitoramento.
- Supervisionar operações.

****Limitações****

- NÃO aprova licenças.
- NÃO aprova produtos.
- NÃO aprova price proposals.
- NÃO edita dados críticos.
- NÃO toma decisões estratégicas.

****2.4.4 SPECIALIST (Especialista Governamental)****

****Resumo****

Papel técnico responsável por análises de mercado e criação de price proposals.

****Responsabilidades****

- Analisar dados de mercado e tendências.
- Criar price proposals (rascunho → submetida).
- Revisar propostas rejeitadas.
- Criar relatórios fiscais e operacionais.
- Realizar auditoria inicial de anomalias.
- Analisar pedidos, transações e mercado.

****Limitações****

- Não aprova nada.
- Não publica nada.

- Apenas analisa e produz relatórios.

 **2.4.5 ANALYST (Analista Técnico)**

Resumo

Nível júnior/técnico operacional que auxilia SPECIALIST e STAFF.

Responsabilidades

- Coletar dados.
- Criar relatórios operacionais.
- Gerar gráficos e KPIs.
- Validar integridade de informações.
- Analisar pedidos e transações.

Limitações

- Não toma decisões.
- Não cria price proposals.
- Não altera políticas.

 **2.4.6 COMPLIANCE (Auditoria e Fiscalização)**

Resumo

Camada independente responsável pela integridade do sistema.

Responsabilidades

- Auditar logs imutáveis.
- Investigar transações suspeitas.
- Bloquear transações de risco.
- Emitir relatórios de auditoria.
- Fiscalizar políticas.
- Validar exclusão de usuários STATE/STAFF.
- Monitorar atividades incomuns.

Limitações

- Não opera módulos do governo.
- Não publica produtos.
- Não cria price proposals.

 **2.4.7 CUSTOMS (Aduana)**

Resumo

Valida documentos aduaneiros e embarques cross-border.

Responsabilidades

- Validar documentos fiscais de fronteira.
- Validar certificados de origem.
- Revisar cargas relativas a exportação/importação.

Limitações

- Não edita pedidos.
- Não toma decisões fiscais.
- Não cancela embarques.

2.4.8 COMPANY (Empresa)

Resumo

Perfil empresarial regulado pelo sistema.

Responsabilidades

- Gerenciar informações da empresa.
- Enviar documentação para licenciamento.
- Operar catálogo, pedidos e embarques.
- Cumprir políticas fiscais e de compliance.

Limitações

- Não acessa dados governamentais.
- Não realiza auditorias.

2.4.9 PRODUCER (Fornecedor / Fabricante)

Responsabilidades

- Criar produtos.
- Gerenciar catálogo interno.
- Submeter produtos para revisão.
- Receber notificações de aprovação/rejeição.

2.4.10 BUYER (Comprador)

Responsabilidades

- Criar pedidos.
- Realizar pagamentos.
- Consultar histórico.
- Receber notificações.

2.4.11 OPERATOR (Operador Logístico)

Responsabilidades

- Registrar eventos de embarque.
- Enviar documentos logísticos.
- Atualizar tracking.
- Sincronizar eventos aduaneiros.

2.5 Matriz de Permissões (Resumo)

	Módulo	State	Staff	Specialist	Analyst	Compliance	Company					
	Produtor	Buyer										
	-----	-----	-----	-----	-----	-----	-----					
	-----	-----										
	Licenças	✔	⚠ Valida	✖	✖	✔	Envia	✖	✖			
	Produtos	✔	Aprova	⚠ Encaminha	✖	✖	✔	✔	✔ Cria	✖		
	Price Proposals	✔	Aprova	✖	✔	Cria	✖	✔	Audita	✖	✖	✖
	Pedidos	✔	Auditor	✔	Visualiza	✔	Analisa	✔	Analisa	✔	Audita	
	✔	✖	✔	Cria								

Transações			Visualiza		Analisa		Analisa		Audita		
Embarques			Visualiza		Analisa			Audita			
Fiscalidade		Cria		Implementa				Audita			

2.6 Conclusão

Este módulo estabelece a base complete de:

- Papéis,
- Autoridade,
- Regras de negócio,
- Escopos de responsabilidade,
- Limitações estruturais,
- Fluxos de aprovação e auditoria.

Servirá para construir:

- ****Módulo 3 – Workflows****
- ****Módulo 4 – Regras de Negócio****
- ****Módulo 5 – APIs****
- ****Módulo 6 – Permissões Strapi****
- ****Módulo 7 – Estrutura das Coleções****

MÓDULO 3.0 — WORKFLOWS OFICIAIS, totalmente estruturado, profissional, e 100% alinhado com as regras de negócio e roles estabelecidos.

Será o documento **mais estratégico** da documentação — ele define como o sistema funciona na prática.

MÓDULO 3.0 — WORKFLOWS OFICIAIS DO SISTEMA

(documentação modular, formato Markdown, pronta para GitHub/Notion)

3.0 Workflows Oficiais

3.1 Introdução

Este módulo descreve todos os fluxos operacionais, regulatórios, fiscais e logísticos do ecossistema governamental do Corredor do Lobito.

Cada workflow descreve:

- Atores envolvidos (roles)
- Sequência de ações
- Regras de negócio aplicáveis
- Decisões e ramificações
- Auditoria
- Integrações entre módulos

Os workflows estão agrupados em:

1. Licenciamento de Empresas
2. Publicação Oficial de Produtos
3. Propostas de Preço (Price Proposal)
4. Processamento de Pedidos (Orders)
5. Transações Financeiras
6. Embarques e Logística
7. Auditoria e Compliance
8. Dashboards e Relatórios

3.2 Workflow 1 — Licenciamento de Empresas

Objetivo

Validar documentação da empresa e aprovar ou rejeitar a licença para operar no ecossistema.

Atores

- COMPANY
- STAFF (validação técnica)
- STATE (decisão)
- COMPLIANCE (auditoria)

Fluxo Detalhado

```
``mermaid
sequenceDiagram
    participant C as Company
    participant ST as Staff
    participant S as State
    participant CP as Compliance

    C->>ST: Envia documentação (POST /companies/:id/documents)
    ST->>ST: Valida tecnicamente (formatos, assinaturas, expiração)
    alt Documentação válida
        ST->>S: Encaminha para decisão (forward_to_state_decision)
        S->>S: Analisa licenciamento
        alt Aprovada
            S->>ST: Ordem: "Aprovar licença"
            ST->>C: Comunica aprovação
        else Rejeitada
            S->>C: Comunica rejeição + motivos
        end
    else Documentação inválida
        ST->>C: Solicita correções
    end

    CP->>S: Auditoria periódica de licenças
```

Regras de Negócio

- STAFF **não** decide aprovação.
- STATE é a autoridade máxima.
- Toda ação de STATE gera Audit Log.
- COMPLIANCE verifica fraudes documentais.

3.3 Workflow 2 — Publicação Oficial de Produtos

Objetivo

Permitir que produtos cadastrados pelo PRODUTOR sejam validados e publicados oficialmente pelo Estado.

Atores

- PRODUCER
- STAFF
- STATE
- COMPLIANCE

Fluxo

```
sequenceDiagram
    participant P as Producer
    participant ST as Staff
    participant S as State
    participant CP as Compliance

    P->>API: Cria produto (status=draft)
    P->>API: Solicita publicação (request_publication)

    API->>ST: Notificação de revisão

    ST->>API: Valida documentação técnica
    ST->>S: Encaminha para aprovação

    S->>API: GET /products/:id
    alt Aprovado
        S->>API: POST /products/:id/approve_publication
        API->>P: Notificação de aprovação
    else Rejeitado
        S->>P: Notificação de rejeição + motivos
    end

    CP->>API: Inspecciona logs e histórico
```

Regras de Negócio

- STAFF **valida tecnicamente**, mas não aprova.
- STATE aprova/rejeita.
- PUBLICADO = status oficial que permite venda.
- Cada ação gera um log imutável.

3.4 Workflow 3 — Propostas de Preço (Price Proposal)

Objetivo

Determinar preços oficiais de mercado para produtos regulados.

Atores

- ANALYST
- SPECIALIST
- STATE

Fluxo

```
sequenceDiagram
    participant A as Analyst
    participant SP as Specialist
    participant S as State

    A->>API: Coleta dados (produtos, transações, pedidos)
    SP->>API: POST /price-proposals (draft)
    SP->>API: PUT /price-proposals/:id (ajustes)

    SP->>API: POST /price-proposals/:id/submit
    API->>S: Notificação de proposta pendente

    S->>API: GET /price-proposals/:id
    alt Aprovada
        S->>API: POST /price-proposals/:id/approve
        API->>System: Gera snapshot imutável
        API->>SP: Notificação
    else Rejeitada
        S->>API: POST /price-proposals/:id/reject
        API->>SP: Feedback para ajustes
    end
end
```

Regras de Negócio

- SPECIALIST cria.
- STATE aprova.
- Snapshot de preço é imutável e usado em pedidos futuros.

3.5 Workflow 4 — Processamento de Pedidos (Orders)

Objetivo

Gerenciar pedidos de compra entre compradores e produtores.

Atores

- BUYER
- PRODUCER

- STATE
- STAFF
- COMPLIANCE

Fluxo

```
sequenceDiagram
    participant B as Buyer
    participant P as Producer
    participant API
    participant S as State
    participant ST as Staff
    participant CP as Compliance

    B->>API: Cria pedido
    API->>P: Notificação de novo pedido

    API->>CP: Análise inicial automatizada (fraude, risco)

    alt Pedido suspeito
        CP->>S: Escala caso
        S->>API: PUT /orders/:id/block
        API->>B: Notificação: Pedido bloqueado
        API->>P: Notificação
    else Pedido normal
        P->>API: Confirma stock/envio
    end

    API->>S: Relatórios de pedidos cross-border
```

Regras de Negócio

- STATE pode bloquear e cancelar.
- STAFF apenas monitora.
- BUYER não pode editar após pagamento.

3.6 Workflow 5 — Transações Financeiras

Objetivo

Gerenciar pagamentos, antifraude e estados financeiros.

Atores

- BUYER
- COMPANY
- STATE
- COMPLIANCE

Fluxo Simplificado

```
sequenceDiagram
    participant B as Buyer
    participant API
    participant CP as Compliance
    participant S as State

    B->>API: Inicia pagamento
    API->>CP: Verificação antifraude
    alt Risco alto
        CP->>S: Solicita bloqueio
        S->>API: PUT /transactions/:id/block
        API->>B: Notificação: transação bloqueada
    else Ok
        API->>API: Confirma pagamento
    end
    end
```

3.7 Workflow 6 — Embarques e Logística

Objetivo

Gerenciar envios logísticos e validações aduaneiras.

Atores

- OPERATOR
- CUSTOMS
- STATE
- STAFF

Fluxo

```
sequenceDiagram
    participant O as Operator
    participant C as Customs
    participant API
```

```

participant S as State

O->>API: Cria embarque
API->>C: Solicita validação aduaneira

C->>API: Aprova ou Rejeita
alt Rejeitado
    API->>O: Bloqueado
else Aprovado
    API->>O: Liberado
end

O->>API: Atualiza tracking a cada evento

```

Regras de Negócio

- CUSTOMS valida documentação de fronteira.
- STATE pode intervir e bloquear.

3.8 Workflow 7 — Auditoria e Compliance

```

sequenceDiagram
    participant CP as Compliance
    participant API
    participant S as State

    CP->>API: GET /logs (filtros: entity, action, period)
    CP->>API: GET /transactions/high-risk
    CP->>API: GET /orders/suspicious
    alt Detecta anomalia
        CP->>S: Envia relatório crítico
        S->>API: Ação (block/suspend)
    else Nada crítico
        CP->>API: Salva relatório monthly
    end
end

```

3.9 Workflow 8 — Dashboards e Relatórios

Atores

- ANALYST
- SPECIALIST
- STATE

Fluxo Resumido

```

flowchart TD
    A[Analyst coleta dados] --> B[Gera gráficos e KPIs]
    B --> C[Specialist consolida relatórios]
    C --> D[Envia para revisão STATE]

```

D --> E[STATE publica relatório oficial]

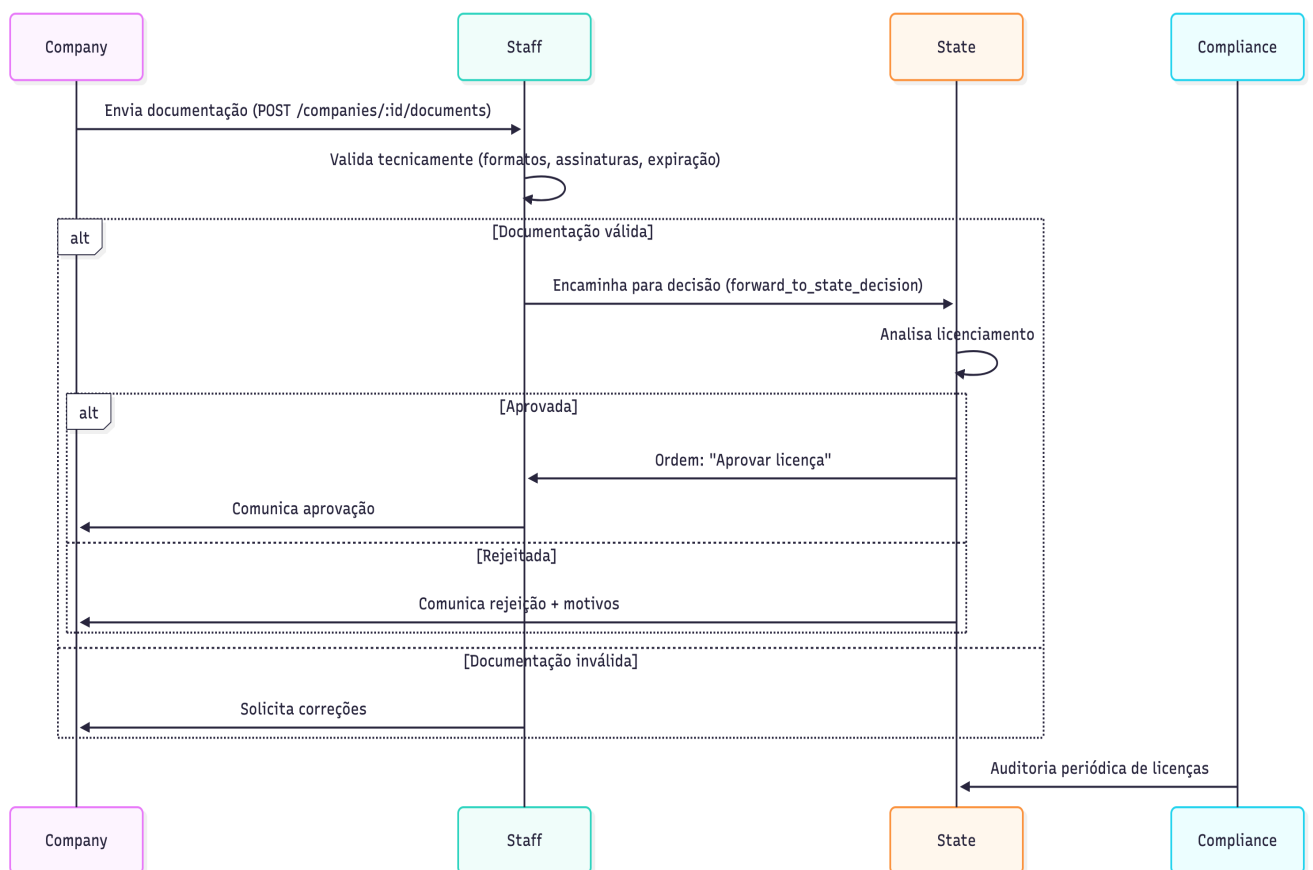


Conclusão

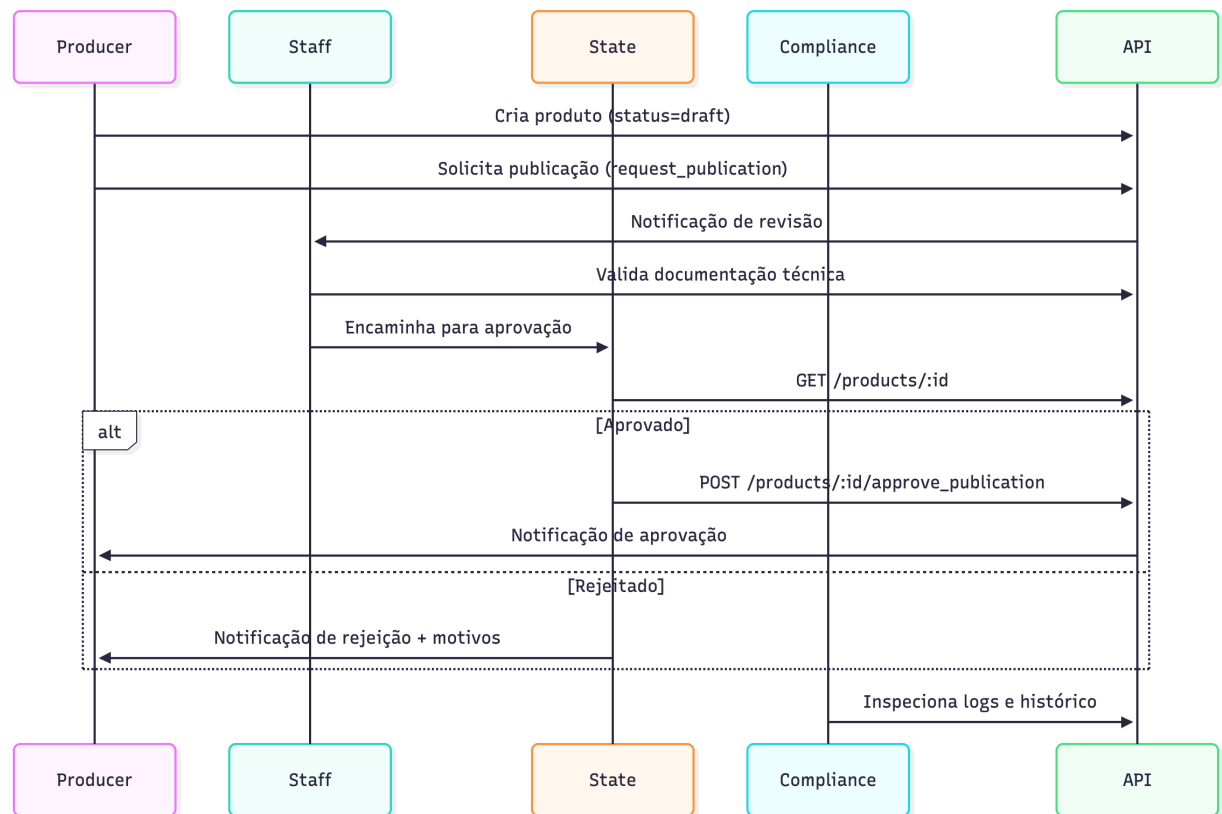
Este módulo define **todos os fluxos operacionais e regulatórios oficiais**, servindo como base para:

- Implementação dos endpoints (Módulo 4)
- Modelagem das coleções Strapi (Módulo 5)
- Regras de acesso e policies (Módulo 6)

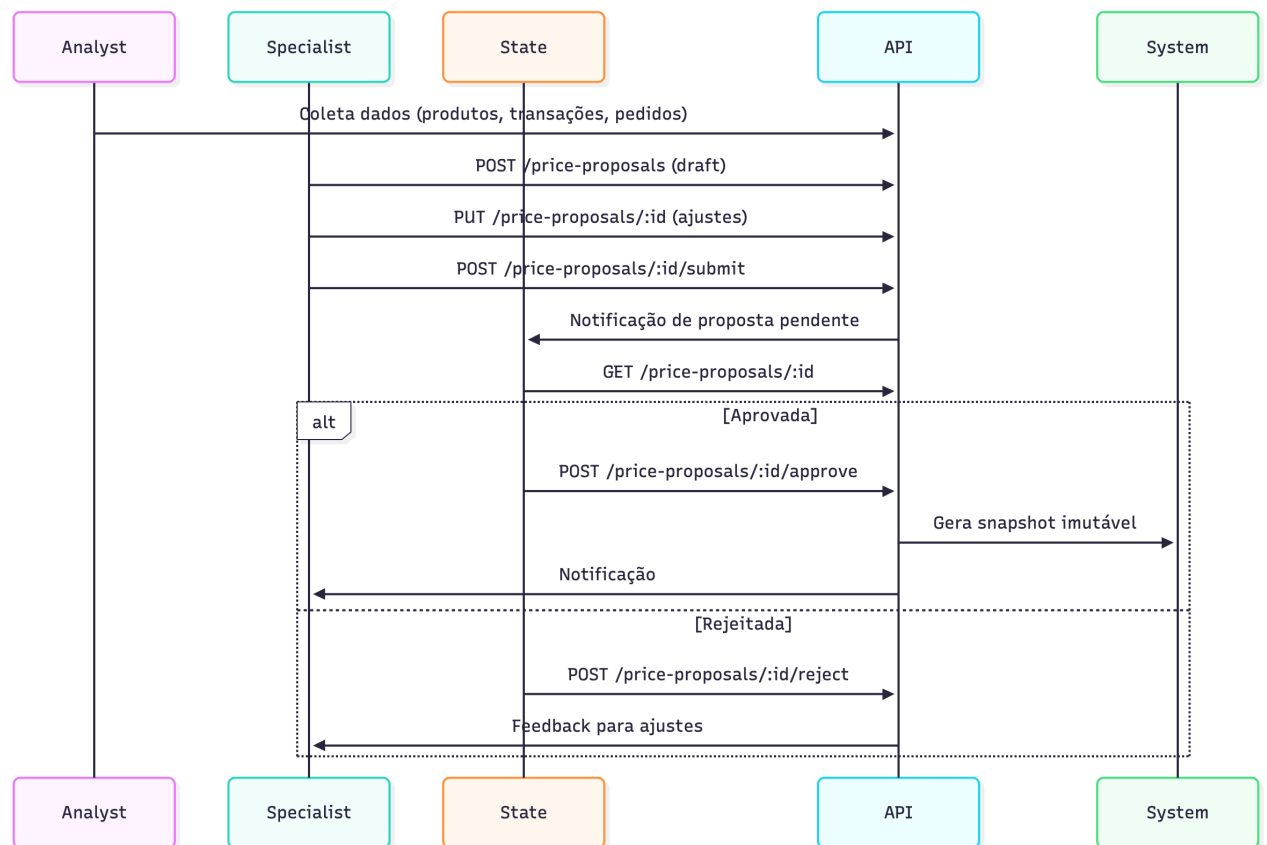
3.2 Workflow 1 – Licenciamento de Empresas



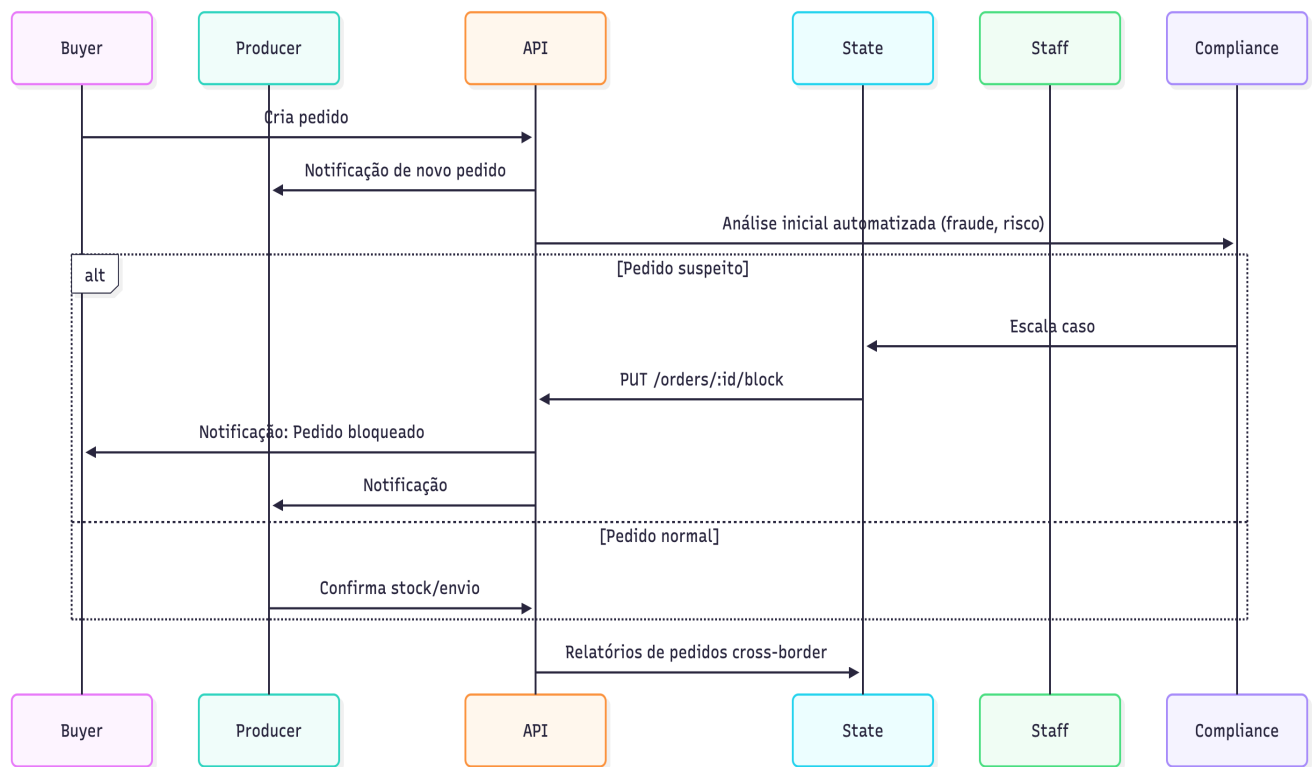
Workflow 2 — Publicação Oficial de Produtos



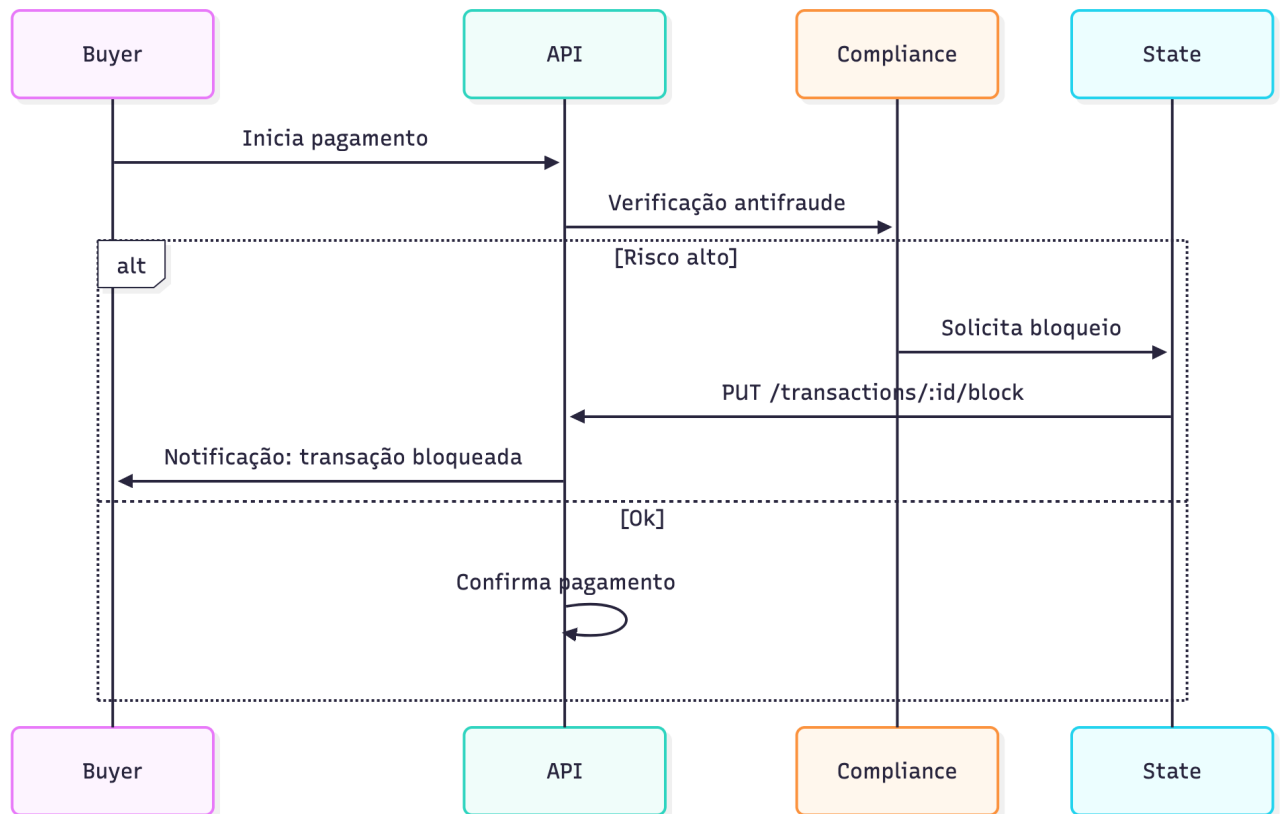
3.4 Workflow 3 — Propostas de Preço (Price Proposal).



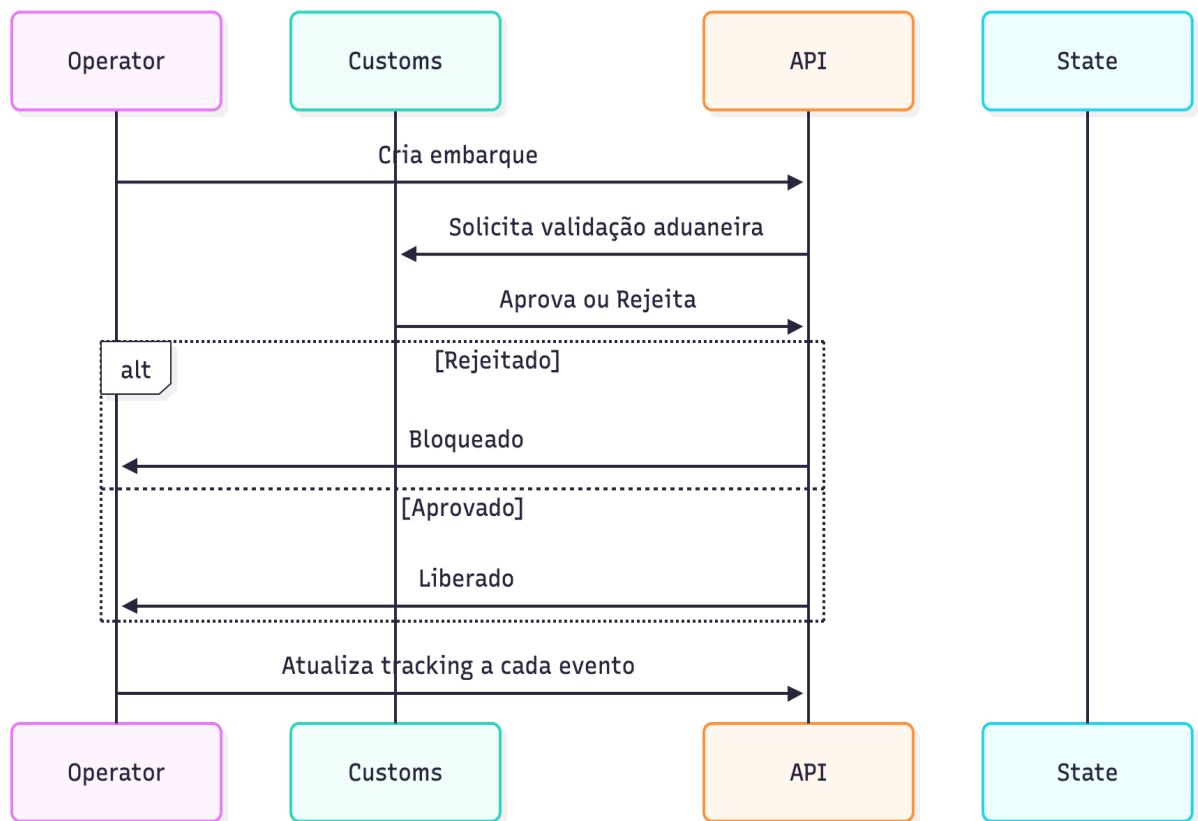
3.5 Workflow 4 — Processamento de Pedidos (Orders).



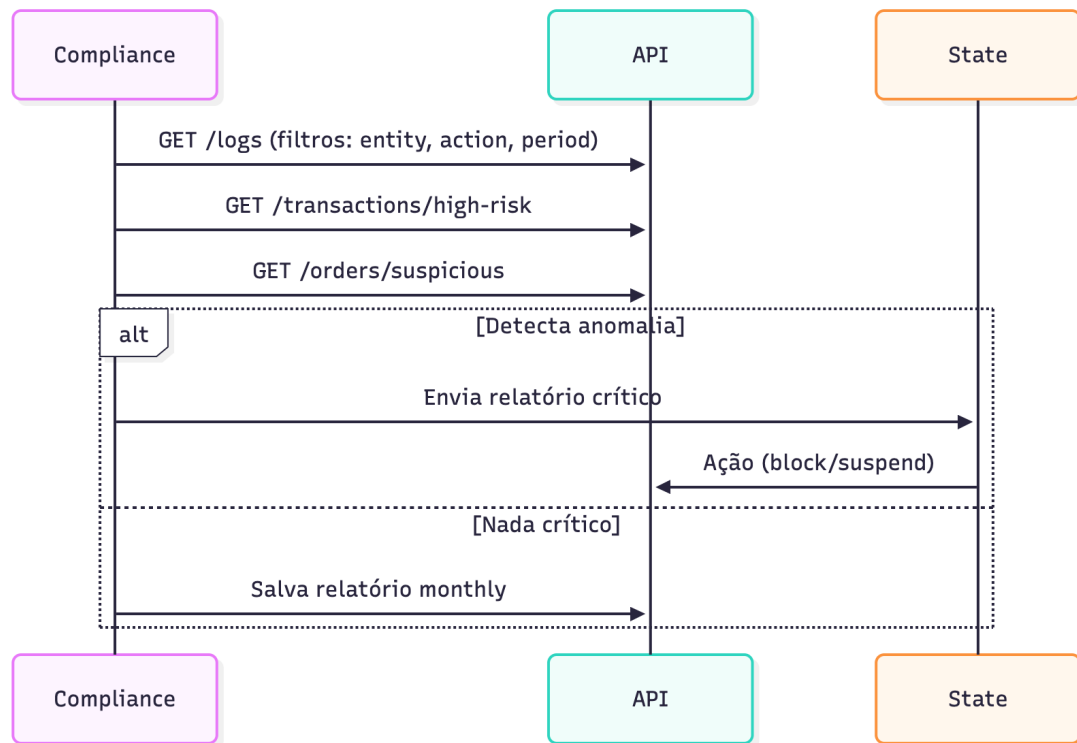
3.6 Workflow 5 — Transações Financeiras.



Embarques e Logística:



3.8 Workflow 7 — Auditoria e Compliance:



3.9 Workflow 8 — Dashboards e Relatórios.



MÓDULO 4 — REGRAS DE NEGÓCIO OFICIAIS, completo, estruturado, totalmente alinhado ao funcionamento real do Ecossistema Governamental do Corredor do Lobito.

Este módulo é **CRÍTICO**, pois contém toda a “Lei Interna” do sistema.

Vai estar 100% pronto para documentação técnica, governança e implementação backend (Strapi / Postgres).

MÓDULO 4 — REGRAS DE NEGÓCIO OFICIAIS

Formato: **Markdown modular** (perfeito para GitHub/Notion)

4.0 Regras de Negócio Oficiais do Sistema

4.1 Introdução

As Regras de Negócio determinam o funcionamento operacional, regulatório, fiscal, logístico e comercial do ecossistema governamental do Corredor do Lobito.

Elas garantem:

- Controle estatal
- Transparência
- Rastreabilidade
- Conformidade entre países
- Fluxo seguro de comércio B2B e B2G
- Auditoria imutável
- Redução de risco e fraude

Cada módulo do sistema possui regras específicas, descritas neste documento.

4.2 Regras Gerais do Ecossistema

4.2.1 Autenticidade e Governança

- Todas as entidades são autenticadas por SSO governamental ou credenciais verificadas.
- STATE é a autoridade máxima do ecossistema.
- Nenhuma entidade pode alterar dados de logs.
- Toda ação sensível gera auditoria (AuditLog).
- Todos os valores financeiros são armazenados com precisão monetária (decimal).

4.2.2 Responsabilidade e Hierarquia

Hierarquia de aprovação:

STATE → SPECIALIST → STAFF → COMPANY → BUYER / OPERATOR / CUSTOMS

- STATE toma decisões finais.
- SPECIALIST consolida análises.
- STAFF executa tecnicamente.
- COMPANY executa operações comerciais.
- BUYER inicia a demanda.
- OPERATOR faz logística.
- CUSTOMS valida fronteiras.

4.3 Regras de Licenciamento de Empresas

4.3.1 Regras de Documentação

- A empresa deve enviar todos os documentos exigidos antes da análise.
- Documentos devem ter:
 - formato válido (PDF/JPEG)
 - assinatura digital válida
 - data de expiração válida
 - carimbo de autenticação

4.3.2 Regras de Validação

- STAFF realiza ****validação técnica****, não aprova.
- STAFF só pode marcar:
 - ``technical_validation = true/false``
 - ``issues_found``
- STAFF não decide licenças.

4.3.3 Regras de Decisão (STATE)

- STATE pode:
 - aprovar licença → ``license_status = "active"``
 - rejeitar → ``license_status = "rejected"``
 - suspender → ``license_status = "suspended"``
 - revogar → ``license_status = "revoked"``

4.3.4 Restrições

- STAFF não pode:
 - alterar `license_status`
 - aprovar empresas
 - emitir licenças
- COMPANY não pode operar sem licença ativa.

4.4 Regras de Produtos (Products)

4.4.1 Criação

- Apenas COMPANY do tipo "producer" cria produtos.
- Produto inicia com ``status = "draft"``.

4.4.2 Publicação

- STAFF valida documentação técnica.
- STATE aprova publicação oficial:
 - ``status = "published_official"``

4.4.3 Suspensão

- STATE pode suspender produtos:
 - fraude documental
 - discrepância fiscal
 - não conformidade regulatória

```
## 4.4.4 Regras de Preço
- COMPANY não define preço final.
- Preço oficial vem de Price Proposal aprovada pelo STATE.
- Preço histórico é imutável:
  - gravado em `price_snapshots`
```

```
# 4.5 Regras de Pedidos (Orders)
```

```
## 4.5.1 Criação
- Somente BUYER pode criar pedidos.
- Total calculado automaticamente:
  - `total_amount = sum(order_lines × price_snapshot)`
```

```
## 4.5.2 Estados Permitidos
```

draft → pending → paid → processing → shipped → delivered

blocked (STATE)

cancelled (STATE)

```
## 4.5.3 Bloqueio de Pedido
- STATE pode bloquear:
  - pedidos incompatíveis
  - valores discrepantes
  - risco de lavagem
  - fraude fiscal
- STAFF não pode bloquear.
```

```
## 4.5.4 Cancelamento
- Apenas STATE pode cancelar pedidos.
- Cancelamento sempre gera audit log.
```

```
# 4.6 Regras de Transações Financeiras
```

```
## 4.6.1 Antifraude (COMPLIANCE)
- Toda transação é analisada por motor antifraude.
- Indicadores de risco:
  - montante acima da média do setor
  - empresa recém-licenciada
  - discrepância entre origem e destino
  - tentativa de bypass fiscal
```

```
## 4.6.2 Decisões
- COMPLIANCE recomenda.
- STATE decide:
  - aprovar
  - bloquear
  - cancelar
```

```
## 4.6.3 Regras Monetárias
- Nenhuma transação pode ser alterada após `status = "completed"`.
- Reembolsos só por decisão STATE.
```

4.7 Regras de Embarques e Logística

4.7.1 Criação

- OPERATOR cria embarques.
- Cada embarque deve ter:
 - número de tracking
 - origem/destino
 - operador logístico
 - documentos anexos

4.7.2 Validação Aduaneira

- CUSTOMS valida documentos:
 - certificados de origem
 - faturas oficiais
 - tarifas aplicáveis

4.7.3 Regras de Fronteira

- CUSTOMS pode:
 - aprovar
 - rejeitar
 - solicitar revisão

4.7.4 Intervenção Estatal

- STATE pode intervir em casos:
 - risco cross-border
 - divergência tarifária
 - contrabando
 - mercadorias proibidas

4.8 Regras de Price Proposals

4.8.1 Criação (SPECIALIST)

- Price Proposal inicia como:
 - `draft`

4.8.2 Submissão

- SPECIALIST envia para STATE:
 - `submitted`

4.8.3 Aprovação (STATE)

- Pode aprovar:
 - `approved`
- Pode rejeitar:
 - `rejected`

4.8.4 Snapshot

- Se aprovado:
 - gerar `price\_snapshot`
 - snapshot é **imutável**
 - não pode ser editado nem pelo STATE

4.9 Regras de Fiscalidade (Taxes)

4.9.1 Regras de Criação

- STATE define impostos oficiais.
- STAFF implementa tecnicamente.

4.9.2 Regras de Validade

- Toda taxa deve ter:
 - `effective\_from`
 - `effective\_to`

4.9.3 Aplicação Automática

- Sistema aplica taxas automaticamente em:
 - pedidos
 - transações
 - embarques cross-border

4.10 Regras de Auditoria

4.10.1 Regras de Logs

- Todos os logs são imutáveis.
- Ninguém pode alterar logs.
- STATE tem acesso total.
- STAFF não pode ver logs do STATE.

4.10.2 Eventos auditados

- mudanças de status
- licenças
- decisões do STATE
- bloqueios
- cancelamentos
- mudanças financeiras
- ações de operadores e aduana

4.11 Regras de Relatórios

4.11.1 Criação

- ANALYST cria relatórios operacionais.
- SPECIALIST cria relatórios fiscais e estratégicos.
- STATE publica oficialmente.

4.11.2 Tipos de Relatórios

- fiscal
- operacional
- auditoria
- risco
- compliance
- comércio exterior

4.12 Regras de Suporte (STAFF)

4.12.1 STAFF pode:

- resolver tickets técnicos
- orientar usuários
- validar documentos
- enviar casos ao STATE

4.12.2 STAFF não pode:

- aprovar licenças
- alterar status críticos
- tomar decisões estratégicas

- cancelar pedidos/transações
- aprovar produtos ou preços

Conclusão

Este módulo estabelece todas as ****normas oficiais**** do sistema governamental do Corredor do Lobito.

Ele é a base para:

- Políticas de acesso (Módulo 5)
 - APIs (Módulo 6)
 - Modelagem Strapi (Módulo 7)
 - Workflows e UI/UX
-

MÓDULO 5 — ENDPOINTS OFICIAIS (API REFERENCE)

✅ Perfeito — **TARIFAS (tariffs)** serão totalmente removidas do ecossistema, da API, das regras e de qualquer área técnica.
O sistema trabalhará apenas com:

- ✅ impostos (taxes)
- ✅ conformidade
- ✅ price proposals
- ✅ restrições aduaneiras via CUSTOMS
- ✅ risk rules
- ❌ **Sem tarifas aduaneiras automáticas.**

Agora eu vou gerar o **MÓDULO 5 — ENDPOINTS OFICIAIS (API REFERENCE)**
100% alinhado ao negócio final, sem tarifas.

✅ MÓDULO 5 — ENDPOINTS OFICIAIS DO ECOSSISTEMA

Formato: **Markdown modular, pronto para Notion / GitHub Wiki / documentação oficial**

```
# 5.0 Endpoints da API (API Reference Oficial)
Documentação oficial dos endpoints REST do Ecossistema Governamental do
Corredor do Lobito.

Todos os endpoints requerem:
- Autenticação JWT + Claims governamentais
- RBAC aplicado via policies
- Auditoria automática

---

# 5.1 Autenticação (Auth)

### POST /api/auth/login
Autenticação via credenciais internas.

### POST /api/auth/oidc/callback
Callback para SSO governamental (OAuth2 / SAML2 / GovID).

### POST /api/auth/2fa/verify
Verificação de segundo fator (obrigatório para STATE, STAFF, SPECIALIST).

### POST /api/auth/logout
```

Encerra sessão autenticada.

5.2 Empresas (Companies)

GET /api/companies
Lista empresas (visão depende do role).

GET /api/companies/:id
Retorna detalhes completos da empresa.

POST /api/companies
Criar empresa (apenas COMPANY via auto-registro controlado).

PUT /api/companies/:id
Atualizar dados permitidos (restrições para STAFF).

POST /api/companies/:id/approve-license
(STATE) Aprovar licença.

POST /api/companies/:id/reject-license
(STATE) Rejeitar licença.

POST /api/companies/:id/suspend
(STATE) Suspender licença.

POST /api/companies/:id/revoke
(STATE) Revogar licença.

POST /api/companies/:id/validate-documentation
(STAFF) Validação técnica dos documentos.

POST /api/companies/:id/forward-to-state-decision
(STAFF) Encaminha documentação validada ao STATE.

5.3 Usuários (Users)

GET /api/users
Somente ADMIN, STATE, STAFF.

GET /api/users/:id
Detalhes do usuário.

POST /api/users
(ADMIN) Criação de usuários governamentais.

PUT /api/users/:id
(ADMIN) Atualizar role ou dados críticos.

PUT /api/users/:id/block
(STATE) Bloquear usuário.

PUT /api/users/:id/role
(ADMIN) Alterar role do usuário.

5.4 Produtos (Products)

```
### GET /api/products
Lista catálogo oficial.

### GET /api/products/:id
Detalhes do produto.

### POST /api/products
(COMPANY produtor) Criar produto (status=draft).

### PUT /api/products/:id
Atualizar dados antes da submissão.

### POST /api/products/:id/request_publication
(COMPANY) Enviar para validação.

### POST /api/products/:id/approve_publication
(STATE) Aprovar publicação → published_official.

### POST /api/products/:id/suspend
(STATE) Suspender produto.
```

5.5 Price Proposals

```
### GET /api/price-proposals
Listar propostas (filtro por role).

### GET /api/price-proposals/:id
Detalhes.

### GET /api/price-proposals/my-proposals
(SPECIALIST) Lista propostas criadas.

### POST /api/price-proposals
(SPECIALIST) Criar proposta (draft).

### PUT /api/price-proposals/:id
(SPECIALIST) Atualizar (draft/rejected).

### POST /api/price-proposals/:id/submit
(SPECIALIST) Submeter para STATE.

### POST /api/price-proposals/:id/approve
(STATE) Aprovar → snapshot gerado.

### POST /api/price-proposals/:id/reject
(STATE) Rejeitar.
```

5.6 Pedidos (Orders)

```
### GET /api/orders
Listagem global (apenas STATE, STAFF, SPECIALIST).

### GET /api/orders/:id
Detalhes completos.

### POST /api/orders
```

(BUYER) Criar pedido.

PUT /api/orders/:id

(BUYER) Permite editar antes do pagamento (status=draft).

POST /api/orders/:id/pay

(BUYER) Realizar pagamento → cria transação.

POST /api/orders/:id/cancel

(STATE) Cancelar pedido.

POST /api/orders/:id/block

(STATE) Bloquear pedido por risco.

POST /api/orders/:id/escalate-to-state

(STAFF) Encaminhar caso.

5.7 Transações (Transactions)

GET /api/transactions

Listagem (STATE, STAFF, SPECIALIST, COMPLIANCE).

GET /api/transactions/:id

Detalhes.

POST /api/transactions

Criada automaticamente via pagamento de order.

POST /api/transactions/:id/block

(STATE) Bloquear transação.

POST /api/transactions/:id/cancel

(STATE) Cancelamento estatal.

GET /api/transactions/summary

Resumo financeiro por país, período, produto.

5.8 Embarques (Shipments)

GET /api/shipments

Listar embarques.

GET /api/shipments/:id

Detalhes com documentos e tracking.

POST /api/shipments

(OPERATOR) Criar embarque.

PUT /api/shipments/:id

(OPERATOR) Atualizar documentos e status interno.

POST /api/shipments/:id/approve

(CUSTOMS) Validar documentos aduaneiros.

POST /api/shipments/:id/reject

(CUSTOMS) Rejeitar entrada ou saída.

POST /api/shipments/:id/hold
(CUSTOMS/STATE) Reter embarque.

5.9 Impostos (Taxes)

GET /api/taxes
Listar impostos.

GET /api/taxes/:id
Detalhes.

POST /api/taxes
(STATE) Criar imposto.

PUT /api/taxes/:id
(STATE) Atualizar regras.

GET /api/taxes/country/:code
Buscar impostos específicos de país.

5.10 Relatórios (Reports)

GET /api/reports
Listar relatórios.

GET /api/reports/:id
Detalhes.

GET /api/reports/my-reports
Listar relatórios do SPECIALIST ou STAFF.

POST /api/reports
Criar relatório.

PUT /api/reports/:id
Atualizar (apenas drafts).

POST /api/reports/:id/submit
Enviar ao STATE.

POST /api/reports/:id/publish
(STATE) Publicar relatório oficial.

GET /api/reports/export-data
Exportação de dados agregados.

5.11 Auditoria (Audit Logs)

GET /api/logs
Listagem de logs (restrições por role).

GET /api/logs/:id
Detalhe.

GET /api/logs/export

Exportação para auditorias.

GET /api/logs/suspicious-activities
(COMPLIANCE/STATE) Acesso a atividades suspeitas.

5.12 Suporte (Support Tickets)

GET /api/support-tickets
(staff/state) Listagem.

GET /api/support-tickets/:id
Detalhes.

POST /api/support-tickets
Criar ticket.

POST /api/support-tickets/:id/resolve
(STAFF) Resolver ticket técnico.

POST /api/support-tickets/:id/escalate
(STAFF) Enviar ao STATE.

5.13 Dashboards e Analytics

GET /api/dashboard
Painel consolidado por role (STATE, STAFF, SPECIALIST).

GET /api/dashboard/metrics
KPIs fiscais, operacionais, logísticos.

GET /api/analytics/revenue
Receita por país / produto / período.

GET /api/analytics/logistics-performance
Performance logística por operador.

GET /api/analytics/compliance-score
Pontuações de conformidade.

MÓDULO 6 — POLÍTICAS DE SEGURANÇA E COMPLIANCE, totalmente alinhado ao ecossistema governamental do Corredor do Lobito.

Este módulo define:

- ✓ Autenticação
- ✓ Autorização (RBAC)
- ✓ Policies
- ✓ Controlo de acesso por role
- ✓ Regras de auditoria
- ✓ Segurança de dados
- ✓ Comunicação segura
- ✓ Ciclo de decisão entre STAFF ↔ STATE ↔ COMPLIANCE
- ✓ Eventual bloqueio automático e manual

Formato 100% profissional: **Markdown modular**.

✓ MÓDULO 6 — POLÍTICAS DE SEGURANÇA E COMPLIANCE

6.0 Políticas de Segurança e Compliance

Este módulo documenta políticas de autenticação, autorização, auditoria, segurança operacional e regras de conformidade do ecossistema governamental do Corredor do Lobito.

A segurança é estruturada em quatro pilares:

1. Autenticação forte (SSO + 2FA)
2. Autorização avançada (RBAC + Policies)
3. Auditoria imutável
4. Conformidade regulatória e antifraude

6.1 Autenticação (Authentication)

6.1.1 Métodos Suportados

- ****SSO Governamental (GovID)****
 - OAuth2
 - OpenID Connect
 - SAML2
- ****Login interno para empresas****
 - Email + password
 - TOTP opcional
- ****2FA obrigatório****
 - STATE
 - STAFF
 - SPECIALIST
 - COMPLIANCE

```
## 6.1.2 Senhas e políticas
- Mínimo 12 caracteres
- Expira automaticamente a cada 90 dias
- Proibido repetir as últimas 3 senhas
- Tentativas falhadas:
  - 5 falhas → bloqueio de 15 minutos
  - 10 falhas → bloqueio manual (STATE)
```

6.2 Autorização (RBAC)

6.2.1 Papéis (Roles)

Role	Nível	Permissões	Descrição
ADMIN	Máximo	Total	Configura sistema e roles
STATE	Alto	Decisão/Regulação	Aprova licenças, preços, cancela/bloqueia
STAFF	Médio	Operacional	Executa decisões do STATE
COMPLIANCE	Alto	Auditoria antifraude	Analisa transações e riscos
SPECIALIST	Médio	Inteligência/Propostas	Cria price proposals e relatórios
ANALYST	Médio	Dados	Relatórios operacionais
COMPANY	Empresa	Comercial	Registra produtos e gera faturas
BUYER	Corporativo	Comercial	Faz pedidos
OPERATOR	Logístico	Transporte	Cria embarques
CUSTOMS	Aduana	Validação	Aprova/rejeita documentos fronteiriços

6.3 Políticas Oficiais

Políticas determinam ****quem pode fazer o quê****, em cada módulo do sistema.

6.3.1 Tipos de Políticas

- ****read-policy****
Limita visualizações.
- ****write-policy****
Controla atualização e criação.
- ****critical-policy****
Restringe ações sensíveis (STATE).
- ****workflow-policy****
Garante a execução correta das etapas.
- ****cross-border-policy****
Valida fluxos internacionais.

6.3.2 Políticas Críticas (Core)

```
### ✅ policy: only_state_can_approve
```

```
if (user.role !== 'state') deny
```

```
### ✅ policy: state_can_block_orders
```


if (user.role !== 'state') deny

✅ policy: staff_cannot_decide_license

if (user.role === 'staff') deny update license_status

✅ policy: snapshot_immutable

if price_snapshot.isFinal = true → deny update/delete

✅ policy: companies_cannot_change_critical_fields

deny fields: license_status, compliance_score, approved_by_state

✅ policy: customs_can_only_act_on_shipments

deny access to modules != shipments

✅ policy: no_write_on_logs

deny update/delete on audit_logs for all roles

✅ policy: buyer_restricted_to_their_orders

orders.company_id must equal buyer.company_id

✅ policy: operator_only_on_shipments

operators cannot access orders, transactions, licenses

6.4 Auditoria (Audit Logs)

6.4.1 Eventos Auditados

Cada uma das ações abaixo é registrada com:

- user_id
- role
- IP
- timestamp
- entity
- before → after
- metadata

Eventos críticos auditados:

- Aprovação/rejeição de licenças
- Ações do STATE

- Alterações fiscais
- Propostas de preço
- Mudanças em pedidos
- Suspensões e bloqueios
- Criação e edição de produtos
- Validação aduaneira
- Logs de transações financeiras
- Erros suspeitos

6.4.2 Propriedades da Auditoria

- ****Imutável**** (append-only)
- ****Não pode ser apagado****
- ****Não pode ser editado****
- ****STATE tem acesso total****
- STAFF não pode ver logs do STATE

6.5 Segurança de Dados

6.5.1 Criptografia

- ****AES-256**** para dados em repouso
- ****TLS 1.3**** para comunicação
- ****HSTS**** ativo
- ****HTTP bloqueado****

6.5.2 Proteção contra acessos indevidos

- Rate limit em endpoints sensíveis
- Proteção contra:
 - SQL Injection
 - XSS
 - CSRF
 - Brute force

6.5.3 Estratégia de Backup

- Backup diário
- Retenção: 5 anos
- Armazenamento multi-país
- Criptografado

6.6 Compliance e Risco

6.6.1 Riscos Monitorados

- Transações de alto valor
- Empresas recém-licenciadas
- Discrepâncias fiscais
- Padrões suspeitos de pedidos
- Tentativas de evasão aduaneira
- Falhas repetidas de login

6.6.2 Decisões de Risco

Papel	Pode recomendar	Pode decidir
COMPLIANCE	✓	✗
STAFF	✓ (encaminhar)	✗
STATE	✓	✓

Apenas ****STATE**** toma decisão final sobre:

- Bloquear pedidos

- Suspender empresas
- Cancelar transações
- Rejeitar embarques
- Avaliação final de fraude

6.7 Policies por Role (Resumo Oficial)

STATE

- Acesso total read/write
- Pode bloquear, suspender, revogar
- Pode aprovar/rejeitar
- Pode publicar relatórios
- Pode cancelar pedidos e transações

STAFF

- Pode validar documentação
- Não pode aprovar nada
- Não pode cancelar/bloquear
- Pode encaminhar ao STATE
- Pode implementar políticas aprovadas

COMPLIANCE

- Leitura total
- Sem permissões de escrita
- Recomendações de risco
- Acesso completo ao antifraude

SPECIALIST

- Cria price proposals
- Cria relatórios fiscais
- Não pode aprovar/rejeitar

ANALYST

- Cria relatórios operacionais
- Acesso de leitura expanded

COMPANY

- Cria produtos
- Atualiza dados próprios
- Não acessa dados de outras empresas

BUYER

- Cria pedidos e paga
- Somente seus pedidos
- Não acessa informações de terceiros

OPERATOR

- Acessa apenas Shipments
- Atualiza status logístico

CUSTOMS

- Acessa apenas Shipments
- Aprova/rejeita documentos de fronteira

6.8 Segurança Internacional

6.8.1 Governança Multi-País

O ecossistema opera em:

- Angola
- RDC
- Zâmbia
- Tanzânia

Cada país possui:

- STATE local
- Regras de validação específicas
- Tratamento fiscal próprio

6.8.2 Regras Cross-Border

- CUSTOMS de cada país vê apenas:
 - embarques relacionados ao país
 - documentos aduaneiros necessários
- STATE pode ver todos os países
- Transações financeiras são individualizadas por país

6.9 Conclusão

Este módulo consolida as **políticas oficiais** do ecossistema, criando a fundação de segurança e confiança do sistema governamental do Corredor do Lobito.



Módulo 6 Pronto.

Agora seguimos para o próximo:



MÓDULO 7 — ESTRUTURA DE DADOS (MODELS HIGH-LEVEL)

Onde vou definir:



Campos



Tipos



Relações



Estrutura oficial de cada entidade



Base para criação dos content-types Strapi

MÓDULO 7 — ESTRUTURA DE DADOS (MODELS HIGH-LEVEL), totalmente alinhado à lógica de negócio e **sem tarifas** (como solicitado).

Este módulo é extremamente importante porque:

- ✓ Define **todas as entidades oficiais do ecossistema**
- ✓ Serve como **referência mestre para os content-types do Strapi**
- ✓ Garante **coerência entre roles, workflows e regras de negócio**
- ✓ Estabelece **IDs, enums, relacionamentos, constraints e campos obrigatórios**

Tudo em **formato profissional**, perfeito para documentação empresarial.

✓ **MÓDULO 7 — ESTRUTURA DE DADOS (MODELS HIGH-LEVEL)**

7.0 Estrutura de Dados (High-Level Models)

Este módulo descreve todos os modelos de dados essenciais para o ecossistema governamental/económico do Corredor do Lobito.

- ✓ Todos os modelos seguem:
 - Padrão relacional (PostgreSQL)
 - Estrutura pensada para Strapi
 - Relações claras (1-N, N-N)
 - Campos críticos protegidos por policies
 - Suporte a auditoria e rastreabilidade

7.1 Users (Usuários)

Representa todos os tipos de utilizadores da plataforma.

Campos Principais

Campo	Tipo	Descrição
id	UID	Identificador
email	string	Login
password	hash	Criptografado
role	enum	admin, state, staff, specialist, analyst, compliance, company, buyer, operator, customs
full_name	string	Nome completo
phone	string	Contacto
status	enum	active, suspended, blocked
company_id	FK (Company)	Apenas para buyer/company/operator
last_login	datetime	Último acesso

Regras

- STATE e ADMIN não pertencem a companies.
- COMPANY, BUYER e OPERATOR pertencem obrigatoriamente a uma empresa.

7.2 Companies (Empresas)

Estrutura empresarial licenciada para operar no corredor.

Campos

Campo	Tipo	Descrição
id	UID	Identificador
name	string	Nome da empresa
country	enum	AGO, DRC, ZMB, TZA
company_type	enum	producer, operator, buyer, mixed
license_status	enum	pending, active, suspended, revoked
license_number	string	Nº único
license_expires_at	datetime	Expiração
address	string	Endereço
contact_email	string	Email
contact_phone	string	Telefone
documentation_validation	JSON	Validação técnica (STAFF)
verified_by_state	boolean	STATE aprovou
approved_by_state_id	FK User	Quem aprovou
created_at	datetime	-
updated_at	datetime	-

Relações

- 1 company → muitos users
- 1 company → muitos products
- 1 company → muitos orders (quando buyer)

7.3 Products (Produtos)

Itens comerciais aprovados pelo governo.

Campos

Campo	Tipo	Descrição
id	UID	
name	string	
description	text	
producer_id	FK Company	
category	string	
price_usd	decimal	
status	enum	draft, pending_review, approved, published_official, rejected
metadata	JSON	Dados adicionais
created_by	FK User	
published_at	datetime	

Regras

- Apenas producer cria
- STATE aprova (approved)
- published_official = produto visível ao público

7.4 Price Proposals (Propostas de Preço)

Propostas técnicas feitas por specialist → aprovadas pelo state.

Campos

Campo	Tipo
-------	------

```
|-----|-----|
| id | UID |
| product_id | FK Product |
| created_by | FK User (specialist) |
| status | enum: draft, submitted, approved, rejected |
| data | JSON | Estrutura analítica |
| snapshot | JSON | Preço final |
| submitted_at | datetime |
| approved_at | datetime |
| approved_by | FK User (state) |
```

Regras

- Apenas STATE aprova
- Snapshot é imutável

7.5 Orders (Pedidos)

Pedidos corporativos entre empresas.

Campos

```
| Campo | Tipo |
|-----|-----|
| id | UID |
| buyer_id | FK Company |
| producer_id | FK Company |
| status | enum: created, paid, blocked, cancelled, processing, shipped,
delivered |
| total_amount | decimal |
| tax_amount | decimal |
| created_by | FK User |
| created_at | datetime |
| updated_at | datetime |
| blocked_reason | string / null |
```

7.6 Order Lines (Itens dos Pedidos)

Campos

```
| Campo | Tipo |
|-----|-----|
| id | UID |
| order_id | FK Order |
| product_id | FK Product |
| qty | integer |
| unit_price | decimal |
| tax_amount | decimal |
```

7.7 Transactions (Transações Financeiras)

Campos

```
| Campo | Tipo |
|-----|-----|
| id | UID |
| order_id | FK Order |
| amount | decimal |
| method | enum: wire_mock, escrow |
| status | enum: pending, paid, failed |
```

```
| paid_at | datetime |
| metadata | JSON |
```

7.8 Shipments (Embarques Logísticos)

Campos

```
| Campo | Tipo |
|-----|-----|
| id | UID |
| order_id | FK Order |
| operator_id | FK Company |
| origin | string |
| destination | string |
| status | enum: ready, in_transit, arrived, delivered, blocked |
| eta | datetime |
| documents | JSON |
| last_location | string |
| updated_at | datetime |
```

7.9 Customs Dispatch (Despacho Aduaneiro)

Campos

```
| Campo | Tipo |
|-----|-----|
| id | UID |
| shipment_id | FK Shipment |
| dispatcher_id | FK User (customs) |
| status | enum: pending_validation, validated, rejected |
| notes | text |
| validated_at | datetime |
```

7.10 Taxes (Impostos)

Campos

```
| Campo | Tipo |
|-----|-----|
| id | UID |
| category | string |
| country | enum: AGO, DRC, ZMB, TZA |
| rate | decimal |
| effective_from | datetime |
| effective_to | datetime |
| created_by | FK User |
```

Regras

- Aplicado automaticamente a orders
- STATE cria/atualiza
- STAFF apenas implementa políticas aprovadas

7.11 Reports (Relatórios)

Campos

```
| Campo | Tipo |
```



```
|-----|-----|
| id | UID |
| title | string |
| type | enum: fiscal, operational, audit, risk |
| author_id | FK User |
| period | string |
| content | JSON |
| status | enum: draft, submitted, published |
| submitted_at | datetime |
| published_at | datetime |
| target_audience | enum: state, specialist, compliance |
```

7.12 Audit Logs (Registros de Auditoria)

Campos

```
| Campo | Tipo |
|-----|-----|
| id | UID |
| user_id | FK User |
| role | string |
| action | string |
| entity | string |
| entity_id | UID |
| before_json | JSON |
| after_json | JSON |
| created_at | datetime |
| meta | JSON |
```

Regra: **Imutável**

7.13 Support Tickets (Suporte)

Campos

```
| Campo | Tipo |
|-----|-----|
| id | UID |
| user_id | FK User |
| type | enum: login_issue, documentation_question, operational_help,
investigation |
| status | enum: open, resolved, escalated |
| content | JSON |
| resolved_by_staff | FK User |
| escalated_to_state | FK User |
| created_at | datetime |
| resolved_at | datetime |
```

7.14 Compliance Alerts (Alertas de Conformidade)

Campos

```
| Campo | Tipo |
|-----|-----|
| id | UID |
| alert_type | enum: high_value_order, suspicious_transaction,
expired_license, blocked_order |
| severity | enum: low, medium, high, critical |
```

```
| entity_id | UID |
| entity_type | string |
| detected_by | FK User (compliance/system) |
| created_at | datetime |
| metadata | JSON |
```

7.15 Dashboard Widgets

Campos

```
| Campo | Tipo |
|-----|-----|
| id | UID |
| title | string |
| kpi_key | string |
| config | JSON |
| created_by | FK User |
| updated_at | datetime |
```

7.16 Conclusão

Este módulo define toda a **arquitetura de dados base** usada no ecossistema do Corredor do Lobito, servindo como referência direta para:

-  criação dos content-types do Strapi
 -  integração com workflows
 -  definição de policies
 -  modelos para API REST/GraphQL
-

Parte Técnica:

perfeito — vou seguir com **Strapi** (implementação dos content-types, fase núcleo), em **documentação modular (Markdown)**, com **arquitetura de pastas + arquivos por tipo** prontos para copiar para `/src/api`.
(sem “tariffs”, conforme sua regra de negócio)

✓ MÓDULO STRAPI — IMPLEMENTAÇÃO DOS CONTENT- TYPES (FASE NÚCLEO)

Objetivo desta fase: ter a espinha dorsal do ecossistema funcionando no Strapi v4: empresas, produtos, propostas de preço, pedidos, transações, embarques, aduana, impostos, relatórios e auditoria — com rotas customizadas essenciais e policies-chave.

1) Estrutura de pastas (para `/src/api`)

```
src/  
  api/  
    company/  
      content-types/company/schema.json  
      controllers/company.js  
      routes/company.js  
      services/company.js  
      policies/  
        only-state-can-decide-license.js  
        staff-can-only-validate-docs.js  
    product/  
      content-types/product/schema.json  
      controllers/product.js  
      routes/product.js  
      services/product.js  
      policies/  
        only-state-can-approve-product.js  
    price-proposal/  
      content-types/price-proposal/schema.json  
      controllers/price-proposal.js  
      routes/price-proposal.js  
      services/price-proposal.js  
      policies/  
        only-specialist-can-submit.js  
        only-state-can-approve.js  
        snapshot-immutable.js  
    order/  
      content-types/order/schema.json  
      controllers/order.js  
      routes/order.js
```

```

    services/order.js
    policies/
      only-state-can-block-or-cancel.js
      buyer-can-only-own-orders.js
    order-line/
      content-types/order-line/schema.json
    transaction/
      content-types/transaction/schema.json
      controllers/transaction.js
      routes/transaction.js
      services/transaction.js
      policies/
        only-state-can-block-transaction.js
    shipment/
      content-types/shipment/schema.json
      controllers/shipment.js
      routes/shipment.js
      services/shipment.js
      policies/
        customs-only-on-shipments.js
    customs-dispatch/
      content-types/customs-dispatch/schema.json
      controllers/customs-dispatch.js
      routes/customs-dispatch.js
      services/customs-dispatch.js
    tax/
      content-types/tax/schema.json
      controllers/tax.js
      routes/tax.js
      services/tax.js
      policies/
        only-state-can-manage-taxes.js
    report/
      content-types/report/schema.json
      controllers/report.js
      routes/report.js
      services/report.js
    audit-log/
      content-types/audit-log/schema.json
      controllers/audit-log.js
      routes/audit-log.js
      services/audit-log.js
      policies/
        no-write-on-logs.js
    support-ticket/
      content-types/support-ticket/schema.json
    compliance-alert/
      content-types/compliance-alert/schema.json
    dashboard-widget/
      content-types/dashboard-widget/schema.json
    extensions/
      users-permissions/
        config/roles-bootstrap.js    # (opcional) seed de papéis/permisões
  middlewares/
    rbac-claims.js                  # injeta claims (role, company_id)

```

2) Content-types essenciais (schemas)

Observação: Todos os schemas abaixo são Strapi v4 (JSON). Você pode copiar arquivo-a-arquivo para as respectivas pastas.

2.1 company/schema.json

```
{
  "collectionName": "companies",
  "info": { "singularName": "company", "pluralName": "companies",
"displayName": "Company" },
  "options": { "draftAndPublish": false },
  "attributes": {
    "name": { "type": "string", "required": true },
    "country": { "type": "enumeration", "enum": ["AGO","DRC","ZMB","TZA"],
"required": true },
    "company_type": { "type": "enumeration", "enum":
["producer","operator","buyer","mixed"], "required": true },
    "license_status": { "type": "enumeration", "enum":
["pending","active","suspended","revoked"], "default": "pending" },
    "license_number": { "type": "string", "unique": true },
    "license_expires_at": { "type": "datetime" },
    "address": { "type": "text" },
    "contact_email": { "type": "email" },
    "contact_phone": { "type": "string" },
    "documentation_validation": { "type": "json" },
    "verified_by_state": { "type": "boolean", "default": false },
    "approved_by_state_id": { "type": "relation", "relation": "oneToOne",
"target": "plugin::users-permissions.user" },
    "products": { "type": "relation", "relation": "oneToMany", "target":
"api::product.product", "mappedBy": "producer" }
  }
}
```

2.2 product/schema.json

```
{
  "collectionName": "products",
  "info": { "singularName": "product", "pluralName": "products",
"displayName": "Product" },
  "options": { "draftAndPublish": false },
  "attributes": {
    "name": { "type": "string", "required": true },
    "description": { "type": "text" },
    "producer": { "type": "relation", "relation": "manyToOne", "target":
"api::company.company", "inversedBy": "products", "required": true },
    "category": { "type": "string" },
    "price_usd": { "type": "decimal", "default": 0, "min": 0 },
    "status": {
      "type": "enumeration",
      "enum":
["draft","pending_review","approved","published_official","rejected"],
      "default": "draft"
    },
    "metadata": { "type": "json" },
    "created_by_user": { "type": "relation", "relation": "oneToOne",
"target": "plugin::users-permissions.user" },
    "published_at": { "type": "datetime" }
  }
}
```

2.3 price-proposal/schema.json

```
{
  "collectionName": "price_proposals",
  "info": { "singularName": "price-proposal", "pluralName": "price-
proposals", "displayName": "Price Proposal" },
  "options": { "draftAndPublish": false },
  "attributes": {
    "product": { "type": "relation", "relation": "manyToOne", "target":
"api::product.product", "required": true },
    "created_by": { "type": "relation", "relation": "oneToOne", "target":
"plugin::users-permissions.user", "required": true },
    "status": { "type": "enumeration", "enum":
["draft","submitted","approved","rejected"], "default": "draft" },
    "data": { "type": "json" },
    "snapshot": { "type": "json" },
    "submitted_at": { "type": "datetime" },
    "approved_at": { "type": "datetime" },
    "approved_by": { "type": "relation", "relation": "oneToOne", "target":
"plugin::users-permissions.user" }
  }
}
```

2.4 order/schema.json

```
{
  "collectionName": "orders",
  "info": { "singularName": "order", "pluralName": "orders", "displayName":
"Order" },
  "options": { "draftAndPublish": false },
  "attributes": {
    "buyer": { "type": "relation", "relation": "manyToOne", "target":
"api::company.company", "required": true },
    "producer": { "type": "relation", "relation": "manyToOne", "target":
"api::company.company", "required": true },
    "status": {
      "type": "enumeration",
      "enum":
["created","paid","processing","shipped","delivered","blocked","cancelled"]
    },
    "default": "created"
  },
  "total_amount": { "type": "decimal", "default": 0 },
  "tax_amount": { "type": "decimal", "default": 0 },
  "blocked_reason": { "type": "string" },
  "lines": { "type": "relation", "relation": "oneToMany", "target":
"api::order-line.order-line", "mappedBy": "order" }
}
```

2.5 order-line/schema.json

```
{
  "collectionName": "order_lines",
  "info": { "singularName": "order-line", "pluralName": "order-lines",
"displayName": "Order Line" },
  "options": { "draftAndPublish": false },
  "attributes": {
```

```

    "order": { "type": "relation", "relation": "manyToOne", "target":
"api::order.order", "inversedBy": "lines", "required": true },
    "product": { "type": "relation", "relation": "oneToOne", "target":
"api::product.product", "required": true },
    "qty": { "type": "integer", "min": 1, "required": true },
    "unit_price": { "type": "decimal", "required": true },
    "tax_amount": { "type": "decimal", "default": 0 }
  }
}

```

2.6 transaction/schema.json

```

{
  "collectionName": "transactions",
  "info": { "singularName": "transaction", "pluralName": "transactions",
"display": "Transaction" },
  "options": { "draftAndPublish": false },
  "attributes": {
    "order": { "type": "relation", "relation": "oneToOne", "target":
"api::order.order", "required": true },
    "amount": { "type": "decimal", "required": true },
    "method": { "type": "enumeration", "enum": ["wire_mock", "escrow"],
"default": "wire_mock" },
    "status": { "type": "enumeration", "enum": ["pending", "paid", "failed"],
"default": "pending" },
    "paid_at": { "type": "datetime" },
    "metadata": { "type": "json" }
  }
}

```

2.7 shipment/schema.json

```

{
  "collectionName": "shipments",
  "info": { "singularName": "shipment", "pluralName": "shipments",
"display": "Shipment" },
  "options": { "draftAndPublish": false },
  "attributes": {
    "order": { "type": "relation", "relation": "manyToOne", "target":
"api::order.order", "required": true },
    "operator": { "type": "relation", "relation": "manyToOne", "target":
"api::company.company", "required": true },
    "origin": { "type": "string", "required": true },
    "destination": { "type": "string", "required": true },
    "status": { "type": "enumeration", "enum":
["ready", "in_transit", "arrived", "delivered", "blocked"], "default": "ready"
},
    "eta": { "type": "datetime" },
    "documents": { "type": "json" },
    "last_location": { "type": "string" }
  }
}

```

2.8 customs-dispatch/schema.json

```

{
  "collectionName": "customs_dispatches",
  "info": { "singularName": "customs-dispatch", "pluralName": "customs-
dispatches", "display": "Customs Dispatch" },

```

```

    "options": { "draftAndPublish": false },
    "attributes": {
      "shipment": { "type": "relation", "relation": "oneToOne", "target":
"api::shipment.shipment", "required": true },
      "dispatcher": { "type": "relation", "relation": "oneToOne", "target":
"plugin::users-permissions.user", "required": true },
      "status": { "type": "enumeration", "enum":
["pending_validation","validated","rejected"], "default":
"pending_validation" },
      "notes": { "type": "text" },
      "validated_at": { "type": "datetime" }
    }
  }
}

```

2.9 tax/schema.json

```

{
  "collectionName": "taxes",
  "info": { "singularName": "tax", "pluralName": "taxes", "displayName":
"Tax" },
  "options": { "draftAndPublish": false },
  "attributes": {
    "category": { "type": "string", "required": true },
    "country": { "type": "enumeration", "enum": ["AGO","DRC","ZMB","TZA"],
"required": true },
    "rate": { "type": "decimal", "required": true },
    "effective_from": { "type": "datetime", "required": true },
    "effective_to": { "type": "datetime" },
    "created_by": { "type": "relation", "relation": "oneToOne", "target":
"plugin::users-permissions.user", "required": true }
  }
}

```

2.10 report/schema.json

```

{
  "collectionName": "reports",
  "info": { "singularName": "report", "pluralName": "reports",
"displayName": "Report" },
  "options": { "draftAndPublish": false },
  "attributes": {
    "title": { "type": "string", "required": true },
    "type": { "type": "enumeration", "enum":
["fiscal","operational","audit","risk"], "required": true },
    "author": { "type": "relation", "relation": "oneToOne", "target":
"plugin::users-permissions.user", "required": true },
    "period": { "type": "string" },
    "content": { "type": "json" },
    "status": { "type": "enumeration", "enum":
["draft","submitted","published"], "default": "draft" },
    "submitted_at": { "type": "datetime" },
    "published_at": { "type": "datetime" },
    "target_audience": { "type": "enumeration", "enum":
["state","specialist","compliance"] }
  }
}

```

2.11 audit-log/schema.json


```
{
  "collectionName": "audit_logs",
  "info": { "singularName": "audit-log", "pluralName": "audit-logs",
"displayName": "Audit Log" },
  "options": { "draftAndPublish": false },
  "attributes": {
    "user": { "type": "relation", "relation": "oneToOne", "target":
"plugin::users-permissions.user" },
    "role": { "type": "string" },
    "action": { "type": "string", "required": true },
    "entity": { "type": "string", "required": true },
    "entity_id": { "type": "string" },
    "before_json": { "type": "json" },
    "after_json": { "type": "json" },
    "meta": { "type": "json" },
    "created_at": { "type": "datetime" }
  }
}
```

2.12 support-ticket/schema.json

```
{
  "collectionName": "support_tickets",
  "info": { "singularName": "support-ticket", "pluralName": "support-
tickets", "displayName": "Support Ticket" },
  "options": { "draftAndPublish": false },
  "attributes": {
    "user": { "type": "relation", "relation": "oneToOne", "target":
"plugin::users-permissions.user", "required": true },
    "type": { "type": "enumeration", "enum":
["login_issue","documentation_question","operational_help","investigation"]
, "required": true },
    "status": { "type": "enumeration", "enum":
["open","resolved","escalated"], "default": "open" },
    "content": { "type": "json" },
    "resolved_by_staff": { "type": "relation", "relation": "oneToOne",
"target": "plugin::users-permissions.user" },
    "escalated_to_state": { "type": "relation", "relation": "oneToOne",
"target": "plugin::users-permissions.user" },
    "resolved_at": { "type": "datetime" }
  }
}
```

2.13 compliance-alert/schema.json

```
{
  "collectionName": "compliance_alerts",
  "info": { "singularName": "compliance-alert", "pluralName": "compliance-
alerts", "displayName": "Compliance Alert" },
  "options": { "draftAndPublish": false },
  "attributes": {
    "alert_type": { "type": "enumeration", "enum":
["high_value_order","suspicious_transaction","expired_license","blocked_ord
er"], "required": true },
    "severity": { "type": "enumeration", "enum":
["low","medium","high","critical"], "required": true },
    "entity_id": { "type": "string", "required": true },
    "entity_type": { "type": "string", "required": true },

```

```

    "detected_by": { "type": "relation", "relation": "oneToOne", "target":
"plugin::users-permissions.user" },
    "metadata": { "type": "json" }
  }
}

```

2.14 dashboard-widget/schema.json

```

{
  "collectionName": "dashboard_widgets",
  "info": { "singularName": "dashboard-widget", "pluralName": "dashboard-
widgets", "displayName": "Dashboard Widget" },
  "options": { "draftAndPublish": false },
  "attributes": {
    "title": { "type": "string", "required": true },
    "kpi_key": { "type": "string", "required": true },
    "config": { "type": "json" },
    "created_by": { "type": "relation", "relation": "oneToOne", "target":
"plugin::users-permissions.user" }
  }
}

```

3) Rotas customizadas essenciais (por arquivo)

3.1 company/routes/company.js

```

'use strict';

module.exports = {
  routes: [
    { method: 'POST', path: '/companies/:id/validate-documentation',
handler: 'company.validateDocumentation', config: { policies:
['api::company.staff-can-only-validate-docs'] } },
    { method: 'POST', path: '/companies/:id/forward-to-state-decision',
handler: 'company.forwardToStateDecision', config: { policies:
['api::company.staff-can-only-validate-docs'] } },
    { method: 'POST', path: '/companies/:id/approve-license', handler:
'company.approveLicense', config: { policies: ['api::company.only-state-
can-decide-license'] } },
    { method: 'POST', path: '/companies/:id/reject-license', handler:
'company.rejectLicense', config: { policies: ['api::company.only-state-can-
decide-license'] } },
    { method: 'POST', path: '/companies/:id/suspend', handler:
'company.suspend', config: { policies: ['api::company.only-state-can-
decide-license'] } },
    { method: 'POST', path: '/companies/:id/revoke', handler:
'company.revoke', config: { policies: ['api::company.only-state-can-decide-
license'] } }
  ]
};

```

3.2 product/routes/product.js

```

'use strict';

module.exports = {

```

```

    routes: [
      { method: 'POST', path: '/products/:id/request_publication', handler:
'product.requestPublication' },
      { method: 'POST', path: '/products/:id/approve_publication', handler:
'product.approvePublication', config: { policies: ['api::product.only-
state-can-approve-product'] } },
      { method: 'POST', path: '/products/:id/suspend', handler:
'product.suspend', config: { policies: ['api::product.only-state-can-
approve-product'] } }
    ]
  };

```

3.3 price-proposal/routes/price-proposal.js

```

'use strict';

module.exports = {
  routes: [
    { method: 'POST', path: '/price-proposals/:id/submit', handler: 'price-
proposal.submit', config: { policies: ['api::price-proposal.only-
specialist-can-submit'] } },
    { method: 'POST', path: '/price-proposals/:id/approve', handler:
'price-proposal.approve', config: { policies: ['api::price-proposal.only-
state-can-approve', 'api::price-proposal.snapshot-immutable'] } },
    { method: 'POST', path: '/price-proposals/:id/reject', handler: 'price-
proposal.reject', config: { policies: ['api::price-proposal.only-state-can-
approve'] } }
  ]
};

```

3.4 order/routes/order.js

```

'use strict';

module.exports = {
  routes: [
    { method: 'POST', path: '/orders/:id/pay', handler: 'order.pay' },
    { method: 'POST', path: '/orders/:id/cancel', handler: 'order.cancel',
config: { policies: ['api::order.only-state-can-block-or-cancel'] } },
    { method: 'POST', path: '/orders/:id/block', handler: 'order.block',
config: { policies: ['api::order.only-state-can-block-or-cancel'] } }
  ]
};

```

3.5 transaction/routes/transaction.js

```

'use strict';

module.exports = {
  routes: [
    { method: 'POST', path: '/transactions/:id/block', handler:
'transaction.block', config: { policies: ['api::transaction.only-state-can-
block-transaction'] } }
  ]
};

```

3.6 shipment/routes/shipment.js

```
'use strict';

module.exports = {
  routes: [
    { method: 'POST', path: '/shipments/:id/approve', handler:
'shipment.approve', config: { policies: ['api::shipment.customs-only-on-
shipments'] } },
    { method: 'POST', path: '/shipments/:id/reject', handler:
'shipment.reject', config: { policies: ['api::shipment.customs-only-on-
shipments'] } },
    { method: 'POST', path: '/shipments/:id/hold', handler:
'shipment.hold', config: { policies: ['api::shipment.customs-only-on-
shipments'] } }
  ]
};
```

4) Policies (regras de acesso) — exemplos por arquivo

Colocar cada arquivo em `src/api/<modulo>/policies/<nome>.js`.

4.1 `only-state-can-decide-license.js`

```
'use strict';

module.exports = async (ctx, next) => {
  const role = ctx?.state?.user?.role;
  if (role !== 'state') return ctx.unauthorized('Only STATE can decide
license.');
```

```
  await next();
};
```

4.2 `staff-can-only-validate-docs.js`

```
'use strict';

module.exports = async (ctx, next) => {
  const role = ctx?.state?.user?.role;
  if (role !== 'staff' && role !== 'state') return ctx.unauthorized('Only
STAFF/STATE can validate or forward.');
```

```
  await next();
};
```

4.3 `only-state-can-approve-product.js`

```
'use strict';

module.exports = async (ctx, next) => {
  if (ctx?.state?.user?.role !== 'state') return ctx.unauthorized('Only
STATE can approve/suspend product.');
```

```
  await next();
};
```

4.4 `only-specialist-can-submit.js`

```
'use strict';
```

```

module.exports = async (ctx, next) => {
  if (ctx?.state?.user?.role !== 'specialist') return
  ctx.unauthorized('Only SPECIALIST can submit.');
```

await next();

```

};
```

4.5 only-state-can-approve.js

```

'use strict';

module.exports = async (ctx, next) => {
  if (ctx?.state?.user?.role !== 'state') return ctx.unauthorized('Only
STATE can approve/reject.');
```

await next();

```

};
```

4.6 snapshot-immutable.js

```

'use strict';

module.exports = async (ctx, next) => {
  // proteção extra; no approve será escrito snapshot e depois bloqueado
  via service/lifecycle
  await next();
};
```

4.7 only-state-can-block-or-cancel.js

```

'use strict';

module.exports = async (ctx, next) => {
  if (ctx?.state?.user?.role !== 'state') return ctx.unauthorized('Only
STATE can block/cancel orders.');
```

await next();

```

};
```

4.8 buyer-can-only-own-orders.js

```

'use strict';

module.exports = async (ctx, next) => {
  const user = ctx.state.user;
  if (user?.role !== 'buyer') return ctx.unauthorized('Only BUYER context
allowed.');
```

// extra: conferir se a order pertence ao company do buyer em
controllers/order.js

```

  await next();
};
```

4.9 only-state-can-block-transaction.js

```

'use strict';

module.exports = async (ctx, next) => {
  if (ctx?.state?.user?.role !== 'state') return ctx.unauthorized('Only
STATE can block transactions.');
```

```
    await next();
  };
```

4.10 customs-only-on-shipments.js

```
'use strict';

module.exports = async (ctx, next) => {
  const role = ctx?.state?.user?.role;
  if (role !== 'customs' && role !== 'state') return ctx.unauthorized('Only CUSTOMS/STATE can act on shipments.');
```

```
  await next();
};
```

4.11 no-write-on-logs.js

```
'use strict';

module.exports = async (ctx, next) => {
  return ctx.unauthorized('Audit logs are append-only (no write/update/delete via API).');
```

```
};
```

5) Controllers — pontos críticos (exemplos)

Obs.: Use `strapi.entityService` e sempre grave **audit logs** após ação sensível.

5.1 product/controllers/product.js (trechos)

```
'use strict';

module.exports = {
  async requestPublication(ctx) {
    const { id } = ctx.params;
    // valida se criador é producer da company
    const product = await
    strapi.entityService.update('api::product.product', id, {
      data: { status: 'pending_review' }
    });
    return product;
  },

  async approvePublication(ctx) {
    const { id } = ctx.params;
    const user = ctx.state.user;
    const productBefore = await
    strapi.entityService.findOne('api::product.product', id);

    const product = await
    strapi.entityService.update('api::product.product', id, {
      data: { status: 'published_official', published_at: new Date() }
    });

    await strapi.entityService.create('api::audit-log.audit-log', {
      data: {
        user: user?.id,
```

```

        role: user?.role,
        action: 'approve_publication',
        entity: 'product',
        entity_id: id,
        before_json: productBefore,
        after_json: product,
        created_at: new Date()
      }
    });

    return product;
  }
};

```

5.2 price-proposal/controllers/price-proposal.js (trechos)

```

'use strict';

module.exports = {
  async submit(ctx) {
    const { id } = ctx.params;
    const proposal = await strapi.entityService.update('api::price-proposal.price-proposal', id, {
      data: { status: 'submitted', submitted_at: new Date() }
    });
    return proposal;
  },

  async approve(ctx) {
    const { id } = ctx.params;
    const user = ctx.state.user;
    const before = await strapi.entityService.findOne('api::price-proposal.price-proposal', id, { populate: ['product'] });

    // gerar snapshot imutável
    const snapshot = {
      product_id: before.product?.id,
      approved_price_usd: before?.data?.recommended_price_usd,
      approved_at: new Date(),
      approved_by: user?.id
    };

    const after = await strapi.entityService.update('api::price-proposal.price-proposal', id, {
      data: { status: 'approved', snapshot, approved_at: new Date(), approved_by: user?.id }
    });

    await strapi.entityService.create('api::audit-log.audit-log', {
      data: {
        user: user?.id, role: user?.role, action: 'approve_price_proposal',
        entity: 'price-proposal', entity_id: id, before_json: before,
        after_json: after, created_at: new Date()
      }
    });

    return after;
  }
};

```

5.3 order/controllers/order.js (trechos)

```
'use strict';

module.exports = {
  async pay(ctx) {
    const { id } = ctx.params;
    const order = await strapi.entityService.findOne('api::order.order',
id, { populate: ['lines'] });

    // calcular total e impostos (simplificado)
    const total = (order.lines || []).reduce((s, l) => s +
Number(l.unit_price) * l.qty + Number(l.tax_amount || 0), 0);

    const updated = await strapi.entityService.update('api::order.order',
id, {
      data: { status: 'paid', total_amount: total }
    });

    await strapi.entityService.create('api::transaction.transaction', {
      data: { order: id, amount: total, method: 'wire_mock', status:
'paid', paid_at: new Date() }
    });

    return updated;
  },

  async block(ctx) {
    const { id } = ctx.params;
    const user = ctx.state.user;
    const before = await strapi.entityService.findOne('api::order.order',
id);

    const after = await strapi.entityService.update('api::order.order', id,
{
      data: { status: 'blocked', blocked_reason: 'state_block' }
    });

    await strapi.entityService.create('api::audit-log.audit-log', {
      data: {
        user: user?.id, role: user?.role, action: 'block_order',
        entity: 'order', entity_id: id, before_json: before, after_json:
after, created_at: new Date()
      }
    });

    return after;
  }
};
```

6) Middleware de claims (role/company_id)

Arquivo: middlewares/rbac-claims.js

```
'use strict';

module.exports = (config, { strapi }) => {
```



```

return async (ctx, next) => {
  // Exemplo: anexar role/company_id à request, vindos do user
  if (ctx.state.user) {
    ctx.state.user.role = ctx.state.user.role || ctx.state.user?.role; //
    onde você persistir
    ctx.state.user.company_id = ctx.state.user.company_id ||
    ctx.state.user?.company?.id;
  }
  await next();
};
};

```

Registrar em config/middlewares.js:

```

module.exports = [
  'strapi::errors',
  'strapi::security',
  'strapi::cors',
  'strapi::logger',
  'strapi::query',
  'strapi::body',
  'strapi::session',
  'strapi::favicon',
  'strapi::public',
  { name: 'global::rbac-claims' }
];

```

7) Recomendações de bootstrap (roles/permissions)

- Se você usa **Users & Permissions** para auth de API, crie um **seed** em `extensions/users-permissions/config/roles-bootstrap.js` definindo **permissões por role** de API (public, authenticated) e **negando** rotas sensíveis por padrão.
 - As *policies* acima já blindam as rotas críticas.
-

8) Próximos passos (rápidos)

1. Copiar cada **schema.json** para sua pasta.
 2. Copiar **routes**, **controllers**, **policies** conforme acima.
 3. `yarn develop` (ou `npm run develop`) para gerar as tabelas.
 4. Criar **usuários** para roles: `state`, `staff`, `specialist`, `buyer`, `producer`, `operator`, `customs`, `compliance`.
 5. Testar endpoints críticos:
 - Companies → `approve-license`
 - Products → `approve_publication`
 - Price Proposals → `submit` e `approve`
 - Orders → `pay`, `block`
-

9) O que vem depois

- **Módulo 8 — Dashboards e Analytics** (KPIs e endpoints agregados)
- **Módulo 9 — Guidelines de Integração** (Next.js + Clerk/SSO)
- **Módulo 10 — Estratégia de Deploy/DevOps**

quer que eu gere **o módulo 8 (Dashboards & Analytics)** em markdown agora, ou prefere que eu **complete os controladores** dos módulos restantes com os handlers padrão (create/find/findOne/update) + hooks (lifecycles) para auditoria automática?